

Challenge 2 Technical Submission Report

Automated Vulnerability Detection & Remediation in GitHub Actions CI/CD Workflows

Team ID: 147	Target Track: Challenge 2 (GitHub Actions Security)
Members: Dante Aliquere Olivas Huamán & Lely Nicole Fernández Risco	OpenRouter Model: anthropic/claude-3.5-sonnet

1. Executive Summary & Methodology Overview

GitHub Actions has emerged as a critical infrastructure layer in modern software engineering. However, unsafe handling of untrusted context variables (e.g., `github.head_ref`, `github.event.issue.title`) in unquoted `run: shell` steps introduces severe Command Injection vulnerabilities. Our solution (**Q-Suyo-Guard**) provides a fully automated, deterministic static analysis and taint-tracking pipeline coupled with an automated patch generation engine integrated with OpenRouter API for LLM-assisted patch explanations.

Core Design Principles:

- Zero False Positives:** Strict scoping against the official `untrusted_data.csv` catalog.
- Cross-Component SHA Resolution:** Recursively resolves nested composite actions and reusable workflows pinned by 12-character commit SHAs.
- First-Exploitable-Sink Reporting:** Strictly enforces the competition Task 1 detection constraint (reporting the earliest sink per taint flow).
- Complete Flow Remediation:** Enforces Task 2 patching rules by sanitizing both initial and downstream re-consumptions of untrusted variables.

2. Vulnerability Detection Methodology (Task 1)

Our detection engine operates in three systematic stages:

Stage A — Untrusted Context Cataloging: We load `untrusted_data.csv` and compile optimized regular expressions supporting exact contexts as well as wildcard expressions (e.g., `github.event.commits[*].message`).

Stage B — Multi-Component Graph Resolution: When a workflow references external components via `uses:` (e.g., `tj-actions/branch-names@9cd06d955f41`), the loader maps the reference to local vendored paths:

- Composite Actions:* `actions/<owner>/<repo>/<sha[:12]>/[subpath]/action.yml`
- Reusable Workflows:* `reusable_workflows/<owner>/<repo>/<sha[:12]>/.github/workflows/<file>.yml`

Stage C — Taint & Line-Accurate Sink Detection: The scanner parses expressions matching `{{ <expr> }}` inside shell execution blocks. It identifies the origin line (from) and pinpoints the exact starting line of the run: step (to).

3. Automated Patch Generation Methodology (Task 2)

To eliminate code injection while strictly preserving workflow execution logic, our patcher applies the canonical environment variable isolation pattern:

```
Vulnerable Pattern:  
- name: Verify Branch
```

```
run: echo "Running on branch ${github.head_ref}"
```

Sanitized Patch Output:

```
- name: Verify Branch
  env:
    HEAD_REF: ${github.head_ref}
  run: echo "Running on branch $HEAD_REF"
```

Key Patching Guarantees:

- **Unified Diff Format:** Outputs standard, git apply-compatible patch files saved to patches/<sample_id>.patch.
- **Downstream Sanitization:** Identifies and sanitizes secondary re-consumptions of tainted variables (e.g., via \$GITHUB_ENV or outputs) across all steps.
- **Indentation & Syntax Integrity:** Preserves exact YAML line endings, quotes, and structural formatting.

4. OpenRouter API & LLM Configuration

In strict compliance with the organizer specifications:

- **Gateway Endpoint:** <https://openrouter.ai/api/v1/chat/completions>
- **Official Model Identifier:** anthropic/claude-3.5-sonnet
- **Team API Key Configured:** sk-or-v1-aaf1fc3...b71 (Passed via CLI or OPENROUTER_API_KEY environment variable)
- **Role:** Generates precise natural-language explanations of remediated vulnerabilities within the generated patch metadata.

5. Empirical Verification & Conclusion

Our framework was benchmarked against the official training dataset (train.csv). The automated detector achieved high precision and recall, correctly isolating single-file and cross-component vulnerabilities, and generating valid unified diffs that pass git apply verification checks without syntax errors.